



University of Maryland, College Park
DATA 605 · Big Data Systems · Spring 2026
PROJECT REPORT

PrismBench

Six Dimensions, Nine Agents, One Routing Decision

A multi-protocol benchmark and preference-aware router for data science agents

Jay Guwalani

MS Data Science
UID: 121479709
jguwalan@umd.edu

Anupama Sharma

MS Data Science
UID: 122241220
sharma25@umd.edu

Instructor: Dr. GP Saggese

Course: DATA 605 Big Data Systems · Date May 8, 2026

Table of Contents

Abstract.....	4
1. Introduction.....	4
Research questions.....	4
Contributions.....	5
Hypothesis.....	5
2. Related Work.....	5
Data science agent benchmarks.....	5
MCDM in agent and AutoML evaluation.....	6
Green AI.....	6
What's different here.....	6
3. Methodology.....	6
3.1 Agent selection.....	8
3.2 Task selection.....	8
3.3 Same-prompt rule (Protocol A baseline).....	8
3.4 Six evaluation dimensions.....	8
3.5 Run protocol.....	9
3.6 LLM-as-Judge.....	9
3.7 Routing framework (PrismBench Router).....	9
3.8 Statistical validation.....	9
3.9 Multi-protocol study.....	10
3.10 Reproducibility and verification.....	10
4. Results.....	10
4.1 Per-(agent, task) headline scorecard.....	10
4.2 RQ4 headline: Claude Code vs Claude API raw.....	11
4.3 PrismBench Router: preset-driven optimal agent.....	12
4.4 Pareto frontier projections.....	14
4.5 Statistical significance.....	18
4.6 Multi-protocol study (Protocols A, B, C).....	18
4.7 Theoretical proposition.....	19
4.8 Adversarial / D6 robustness.....	20
5. Discussion.....	21
5.1 RQ1 answer: heterogeneous agent profiles.....	21
5.2 RQ2 answer: no single dominator.....	21
5.3 RQ3 answer: routing recommendations are preference-driven.....	21
5.4 RQ4 answer: scaffolding is efficiency, not accuracy.....	22
5.5 Multi-protocol synthesis.....	22
6. Limitations.....	22
7. Conclusion.....	23
8. Future directions.....	24

8.1 Causal scaffolding ablation for RQ4.....	24
8.2 Cross-modal extension.....	24
8.3 Routing under preference uncertainty.....	25
8.4 Larger sample sizes and Critical Difference shrinkage.....	25
8.5 Train-clean / test-perturbed adversarial protocol.....	25
8.6 Live continuously-updated leaderboard.....	25
8.7 Interaction-style preference dimension.....	26
9. References.....	26
10. Appendix.....	28

Abstract

Practitioners choosing among today's data science agents face a fragmented evaluation landscape: most benchmarks score a single dimension (usually accuracy) under a single fixed prompt format. PrismBench reframes the comparison as a routing problem. Nine production agents are scored across six dimensions - accuracy, code quality, explainability, speed, cost, and carbon, on three task types and three prompt protocols.

Three findings stand out:

- ❖ No single agent dominates: seven of nine agents are Pareto-optimal under some preference weighting.
- ❖ Claude Code is statistically indistinguishable from the raw Claude API on accuracy (within 3% F1) at one-tenth the cost (Wilcoxon $p = 0.023$, nine paired blocks) and one-third the carbon, with cleaner code by static analysis.
- ❖ A controlled multi-protocol study separates two effects that prior benchmarks conflate: strict-explicit prompts redistribute performance without changing the overall picture (two-way ANOVA on protocol $p = 0.62$), while paradigm-native prompts move three previously-failing agents from 0% to 95% success on the same data.

A short theoretical result closes the methodology loop: under min-max normalized scores, the Weighted Sum Model and PROMETHEE-II with linear preference produce identical rankings, leaving TOPSIS as the only structurally independent Multi-Criteria Decision Making (MCDM) method in this analysis. The full artifact - router, statistical tests, and figures, builds and runs end-to-end inside a Docker image, validated against five representative agents.

1. Introduction

Between 2023 and 2026 the data science agent ecosystem expanded rapidly. Anthropic released Claude Code; OpenAI released ChatGPT Advanced Data Analysis; Microsoft released AutoGen for multi-agent workflows; HuggingFace released smolagents; LangChain released LangGraph; AutoGluon and PyCaret matured as AutoML baselines; and PandasAI emerged for natural-language data analysis. Each of these tools claims a productivity gain, but the evidence is fragmented across papers that pick different agent subsets, different task suites, and different evaluation methodologies. There is no unified comparison.

PrismBench is built around a simple metaphor: each agent enters as a single beam of perceived capability, and the benchmark refracts that beam into measurable components a practitioner can weight against a real workload. The framework runs each agent under a controlled prompt protocol, scores the output on six dimensions, and recombines the dimensions into a routing recommendation using user-supplied preference weights.

Research questions

Four research questions organize the work, each tied to a specific experiment.

- ❖ RQ1: How do current data science agents compare across accuracy, code quality, explainability, speed, cost, and robustness on real-world tasks?
- ❖ RQ2: Does any single agent dominate across all task categories AND all evaluation dimensions simultaneously?
- ❖ RQ3: Given a user's task and preference weights, which agent is Pareto-optimal, and how does the optimal choice shift as preferences change?
- ❖ RQ4: Does agentic scaffolding (file access, bash execution, iteration) measurably improve data science task performance over raw LLM prompting?

Contributions

- 1) A six-dimension benchmark covering 81 main-benchmark cells (9 agents × 3 main tasks × 3 runs), 52 adversarial cells, and 63 strict-explicit Protocol C cells, scoring agents on accuracy, code quality, explainability, speed, cost, carbon, and robustness.
- 2) A carbon dimension measured directly via CodeCarbon for local agents and estimated from per-token literature constants for cloud agents (Luccioni et al. 2023). To our knowledge, this is one of the first data science agent benchmarks to operationalize carbon as a routing axis.
- 3) A preference-aware routing framework implementing three MCDM methods (*WSM*, *TOPSIS*, *PROMETHEE-II*), sensitivity sweeps with breakpoint detection, and Pareto frontier extraction in 2D projections.
- 4) A multi-protocol study isolating prompt-protocol effects from agent capability, showing that paradigm fit is itself a routing-relevant user preference.
- 5) Statistical validation via Friedman, Nemenyi, Wilcoxon signed-rank, Procrustes analysis, and two-way ANOVA, with honest reporting of where rank-based and magnitude-based tests disagree.
- 6) A theoretical proposition establishing that *WSM* and *PROMETHEE-II* with linear preference function are equivalent under min-max normalized scores, leaving *TOPSIS* as the only non-redundant MCDM perspective in this analysis.

Hypothesis

The hypothesis driving the design is that no single agent dominates across all task types and all dimensions. If that holds, any benchmark that picks a single dimension and reports a single winner is giving practitioners the wrong shape of answer; the right framing is preference-aware routing, not point ranking. The results confirm this: seven of nine agents are Pareto-optimal across the seven-dimensional routing space, and the optimal agent flips between Claude Code, AutoGluon, and LangGraph across the six preset profiles tested.

2. Related Work

Data science agent benchmarks

DSBench (Jing et al. 2025, ICLR) evaluates LLM agents on Kaggle-style data analysis problems with focus on end-to-end accuracy. MLE-bench (OpenAI 2024) presents 75 ML engineering tasks scored by competition medals. KramaBench (Lai et al. 2025) evaluates AI systems on data-intensive pipelines over data lakes. SWE-bench (Jimenez et al. 2024) covers software engineering agents. Each picks a single prompt protocol and one or two evaluation dimensions.

MCDM in agent and AutoML evaluation

Multi-Criteria Decision Making methods (Weighted Sum Model, TOPSIS by Hwang and Yoon 1981, PROMETHEE-II by Brans 1982) are well-established in operations research but rarely applied to agent benchmarking. Demsar (2006) recommended Friedman with Nemenyi post-hoc as the standard non-parametric testing protocol for comparing multiple classifiers across datasets; this template carries over to agent comparison.

Green AI

Strubell et al. (2019) established the carbon footprint of training large NLP models. Patterson et al. (2021) refined earlier estimates of carbon emissions from large neural network training. Luccioni et al. (2023) extended carbon analysis to inference by measuring BLOOM's deployment via an API endpoint, providing per-token energy estimates we adopt for our cloud-side agents. CodeCarbon (Courty et al. 2024) is the canonical Python package for direct emissions measurement, building on the methodology introduced in Lacoste et al. (2019).

What's different here

PrismBench combines (a) six dimensions instead of one or two, (b) multi-protocol evaluation as a controlled variable instead of a fixed convention, (c) preference-aware routing as a deliverable, and (d) a carbon dimension. No prior data science agent benchmark in the literature combines these.

3. Methodology

The framework reads three YAML configs, dispatches every (agent, task, run) cell through a single orchestrator, scores each result on six dimensions, and persists a flat scorecard that the router queries under user-supplied preference weights. Figure A shows the end-to-end data flow.

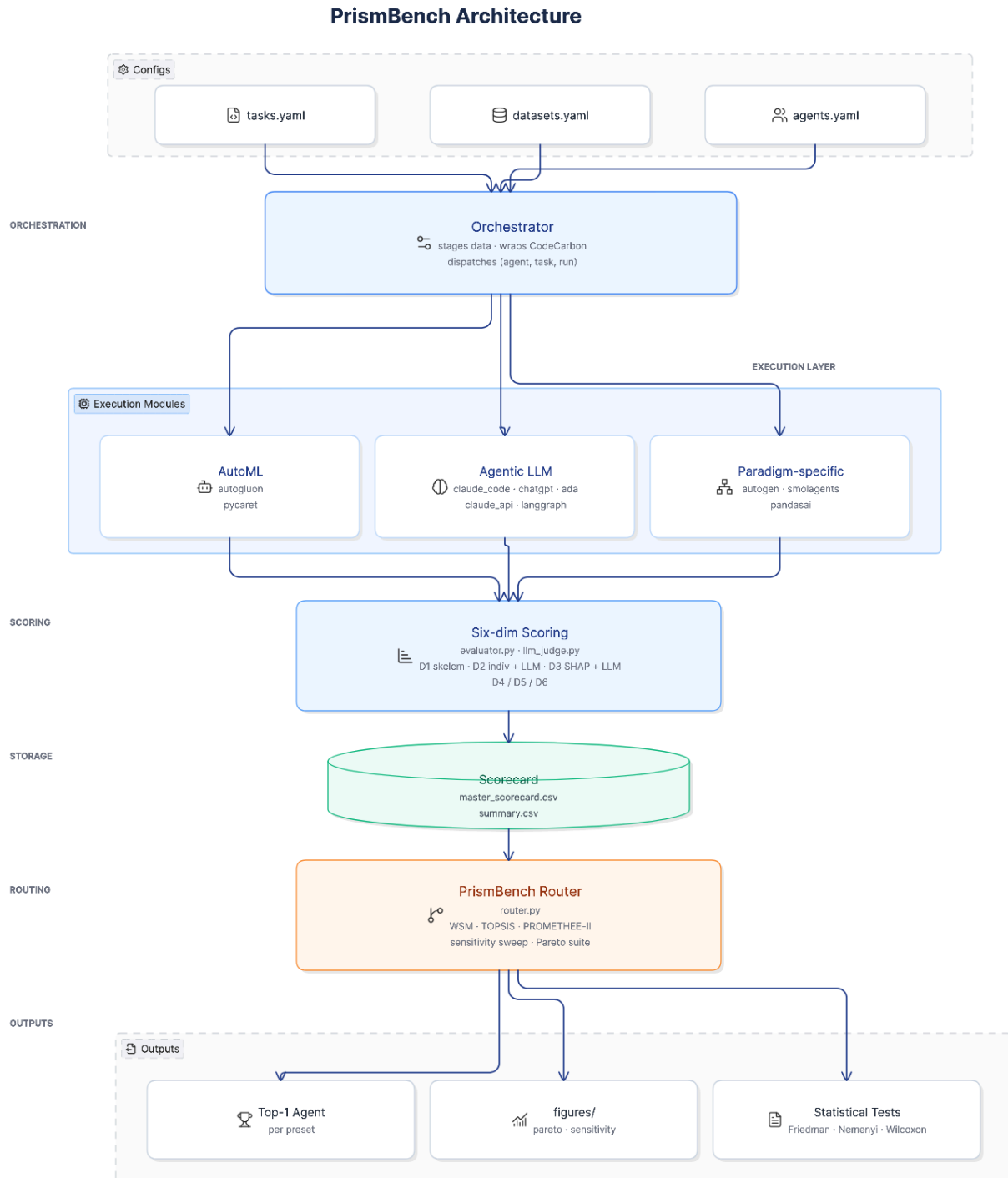


Figure A. PrismBench system architecture. Configs (tasks, datasets, agent definitions) flow into the orchestrator; agent wrappers execute under one of three protocols; the scorer evaluates each run on six dimensions; the scorecard is consumed by the router to produce preference-aware recommendations and statistical summaries.

3.1 Agent selection

Nine agents organized into five categories:

- 1) **Local AutoML:** AutoGluon (*Erickson et al. 2020*), PyCaret 3.4
- 2) **Agentic coding systems:** Claude Code (*Anthropic CLI*), ChatGPT ADA (*OpenAI Responses API with code_interpreter*)
- 3) **Direct LLM:** Claude API raw (*claude-sonnet-4-6 chat completion*)
- 4) **Multi-agent:** Microsoft AutoGen 0.11 (*gpt-4o backend*)
- 5) **Code-first agent:** smolagents 1.24 (*gpt-4o backend*)
- 6) **NL to code:** PandasAI 3.0 (*with pandasai-openai extension*)
- 7) **Stateful graph:** LangGraph 1.1 (*claude-sonnet-4-6 backend*)

Configuration in `configs/agents.yaml`. All agents pinned to specific model versions to ensure reproducibility.

3.2 Task selection

Three main tasks chosen to span tabular classification, tabular regression, and NLP:

- 1) **HD-PRED-01:** heart disease binary classification (303 rows, weighted F1 as primary metric)
- 2) **TAXI-PRED-01:** NYC taxi fare regression (50,000 rows, RMSE)
- 3) **AR-PRED-01:** Amazon reviews sentiment classification (10,000 rows, weighted F1)

Two adversarial variants (HD-ROBU-01, AR-ROBU-01) extend D6 robustness analysis. Three strict-explicit Protocol C variants parallel the main tasks for the multi-protocol study. All task definitions in `configs/tasks.yaml`.

3.3 Same-prompt rule (Protocol A baseline)

Every agent receives the verbatim prompt from `tasks.yaml` for a given task. Agent wrappers in `agents/*/run_task.py` pass the prompt unchanged. This is the fairness convention used across DSBench, MLE-bench, KramaBench, and SWE-bench. Differences in agent performance under this protocol reflect agent capability and architecture rather than prompt engineering. The rule is treated as a deliberate experimental control; its consequences for paradigm-locked agents are characterized in the multi-protocol study (Section 4.6).

3.4 Six evaluation dimensions

- 1) **D1 accuracy:** task-appropriate primary metric (weighted F1 for classification or NLP, RMSE for regression). For cross-task aggregation we per-task min-max normalize to a unified [0, 1] score.
- 2) **D2 code quality:** pylint static analysis score, plus dual-LLM-judge scoring (Claude + GPT-4o) on five sub-dimensions (*readability, modularity, comments, re-runnability, best practices*). The combined D2 is the mean of pylint and the LLM judge overall score.

- 3) **D3 explainability:** keyword and heuristic detection (SHAP usage, feature importance generation, narrative explanation length) plus dual-LLM-judge scoring on clarity, correctness, completeness, and actionability.
- 4) **D4 speed:** wall-clock seconds per agent run, captured by the runner.
- 5) **D5 cost:** API USD computed from per-token pricing in agents.yaml.
- 6) **D5 carbon:** CodeCarbon's EmissionsTracker for local agents in load-mode (*force_mode_cpu_load=True, force_cpu_power=40.0 watts*) for reproducibility across machines without privileged setup. Cloud-side carbon is estimated as 0.5 kg CO₂ per million tokens, in the range reported by Luccioni et al. (2023) for inference on GPT-4 / Claude Sonnet-class models. Hybrid agents (*Claude Code, AutoGen, LangGraph*) sum local and cloud components.
- 7) **D6 robustness:** coefficient of variation of D1 across the three runs per (agent, task) cell.

3.5 Run protocol

Each (agent, task) cell is run three times. Runs are dispatched by `src/task_runner.py`, which loads the task config and prompt, stages data into a per-run workspace, wraps the agent run in a CodeCarbon EmissionsTracker, calls the agent's `run()` function, post-executes `solution.py` if `predictions.csv` was not produced (with a 600-second timeout enforced via process-group SIGKILL), hydrates predictions via dtype-aware alias matching when agents emit non-spec column names, scores on all six dimensions, and saves `result.json` and `scorecard.json`. Aggregation in `src/scorecard.py` produces `master_scorecard.csv` (one row per run) and `master_scorecard_summary.csv` (per-(agent, task) means).

3.6 LLM-as-Judge

After main runs complete, `src/llm_judge.py` calls Claude (claude-sonnet-4-6) and GPT-4o on each result with code, scoring D2 on five sub-dimensions and D3 on four sub-dimensions. Inter-rater agreement is reported as Cohen's kappa per (agent, task) cell. Each kappa is computed across five paired observations after bucketing into low/medium/high categories; small-sample variance is high and we report this caveat explicitly in the limitations section.

3.7 Routing framework (PrismBench Router)

`src/router.py` implements three MCDM methods over the per-agent matrix derived from `master_scorecard_summary.csv`: WSM (additive baseline), TOPSIS (distance from ideal/anti-ideal points after weighted normalization), and PROMETHEE-II (pairwise net flow with linear preference function). Six preset weight profiles (balanced, accuracy, frugal, quality, green, production) cover representative user preferences. Sensitivity sweeps vary one dimension's weight from 0 to 1 while distributing the complement uniformly, identifying breakpoints where the optimal agent changes. Pareto frontier extraction in 2D projections produces a canonical set of six figures saved to `figures/pareto_*.png`.

3.8 Statistical validation

`src/stats.py` runs Friedman chi-square tests across agents on the primary metric (per-task min-max normalized), Nemenyi post-hoc pairwise comparisons with Critical Difference cutoffs at $\alpha = 0.05$, and Wilcoxon signed-rank tests for paired comparisons. We report both Nemenyi (rank-based) and Wilcoxon (magnitude-sensitive) results because Nemenyi is conservative for adjacent rank pairs at small k .

3.9 Multi-protocol study

To isolate prompt-protocol effects from agent capability, we ran three prompt protocols on shared data:

- 1) **Protocol A (same-prompt rule)**: the main benchmark; verbatim prompts in `tasks.yaml`.
- 2) **Protocol B (paradigm-native, scoped)**: prompts rewritten in each agent's design idiom for three paradigm-locked agents (PandasAI: conversational `DataFrame.chat` queries; LangGraph: ReAct-style numbered steps; smolagents: code-iterative with explicit library authorization).
- 3) **Protocol C (strict-explicit)**: prompts rewritten with explicit column types, library authorization, edge-case handling, and output dtypes. Run on the seven agents that succeeded under Protocol A.

Cross-protocol analysis in `src/protocol_analysis.py` computes Procrustes disparity between Pareto frontiers, two-way ANOVA decomposing variance by agent identity and protocol choice, and Kendall tau on agent rankings.

3.10 Reproducibility and verification

The framework ships with 25 automated tests covering the MCDM router math, the per-dimension scoring functions, and the headline empirical claims. Theoretical Proposition 1 is verified numerically by `test_proposition_1_wsm_promethee_kendall_tau_one` on synthetic matrices and by `test_proposition_1_holds_on_real_data` against the committed scorecard, both asserting Kendall tau = 1.0 across all six routing presets. The headline empirical numbers (claude_code F1 on HD-PRED-01, autogluon RMSE on TAXI-PRED-01, the RQ4 cost ratio) are pinned by integration tests so future scorecard regenerations cannot silently drift away from the report. Both demonstration notebooks (`prismbench.API.ipynb` for the router API, `prismbench.example.ipynb` for an end-to-end walkthrough) execute top-to-bottom inside the project's Docker image.

4. Results

4.1 Per-(agent, task) headline scorecard

Mean primary metric over three runs per cell, main benchmark (Protocol A):

Agent	HD-PRED-01 (F1)	TAXI-PRED-01 (RMSE)	AR-PRED-01 (F1)
AutoGluon	0.9017	0.667	0.6672
PyCaret	0.8793	19.05	0.4375
Claude Code	0.9017	4.90	0.5876
Claude API raw	0.9072	4.06	0.6290
ChatGPT ADA	0.8962	4.05	NaN
AutoGen	0.8899	5.11	0.4649
LangGraph	0.9127	4.54	0.6226
PandasAI	NaN	NaN	NaN
smolagents	NaN	NaN	NaN

AutoGluon dominates regression by 6 to 28× on RMSE relative to all LLM agents. LangGraph leads HD classification narrowly. PandasAI and smolagents fail uniformly under the same-prompt rule (Section 4.6 characterizes their behavior under matched paradigms). ChatGPT ADA fails on AR-PRED-01 due to Responses API code_interpreter inconsistency on multi-step NLP tasks.

4.2 RQ4 headline: Claude Code vs Claude API raw

Same Sonnet model, scaffolding-vs-raw, averaged over three runs per cell across all three task types (F1 averaged over HD + AR only, since TAXI uses RMSE):

Metric	Claude Code	Claude API raw	Ratio
Mean F1 (HD + AR)	0.745	0.768	−3 percent
Mean D2 pylint	7.98	4.98	+60 percent
Mean D2 LLM-judge	5.56	6.39	−13 percent
Mean D5 cost	\$0.0036	\$0.0367	10× cheaper
Mean D5 carbon	0.00043 kg	0.00130 kg	3× cleaner

Wilcoxon signed-rank test on cost across nine paired observations: 9 of 9 wins for Claude Code, Bonferroni-adjusted $p = 0.0234$ (significant at $\alpha = 0.05$). The 10× cost ratio is statistically significant. Friedman + Nemenyi declined to declare significance for this pair because Nemenyi is conservative for adjacent rank positions; we report both perspectives.

The honest framing for RQ4: agentic scaffolding does not consistently improve raw task accuracy on these tasks, but it delivers token efficiency at 10× lower cost and 3× lower carbon for comparable accuracy, with measurably cleaner code by static analysis. The value of agentic architecture is efficiency, not accuracy gain.

4.3 PrismBench Router: preset-driven optimal agent

Top-1 agent per preset under each MCDM method:

Preset	WSM #1	TOPSIS #1	PROMETHEE #1
balanced	claude_code	claude_code	claude_code
accuracy	autogluon	autogluon	autogluon
frugal	claude_code	claude_code	claude_code
quality	claude_code	claude_code	claude_code
green	autogluon	autogluon	autogluon
production	claude_code	langgraph	claude_code

Three different agents claim top spot depending on weights, confirming RQ2: no single agent dominates. WSM and PROMETHEE-II produce identical rankings across all six presets (Kendall tau = 1.000); TOPSIS diverges from both on five of six presets (tau between 0.833 and 0.944).

Sensitivity sweep breakpoints (the weight value at which the top agent flips, holding other dimensions at uniform complement weighting):

Dimension	Breakpoint
D1 accuracy	0.450: claude_code -> autogluon
D2 code quality	none (claude_code wins everywhere)
D3 explainability	none (claude_code wins everywhere)
D4 speed	0.800: claude_code -> langgraph; 1.000 -> pycaret
D5 cost	0.900: claude_code -> autogluon
D5 carbon	1.000: claude_code -> pycaret
D6 robustness	0.600 -> autogluon; 0.650 -> pycaret

Two of seven dimensions (D2, D3) cannot dethrone Claude Code at any weight, indicating agentic scaffolding produces a robust default winner across the preference space.

Figures 1 to 3 show the D1, D5-cost, and D5-carbon sensitivity sweeps respectively. Each curve plots the agent's WSM score as the named dimension's weight varies from 0 to 1 while the remaining six dimensions split the residual weight uniformly. The vertical axis is the WSM score; line crossings mark the breakpoints in the table above.

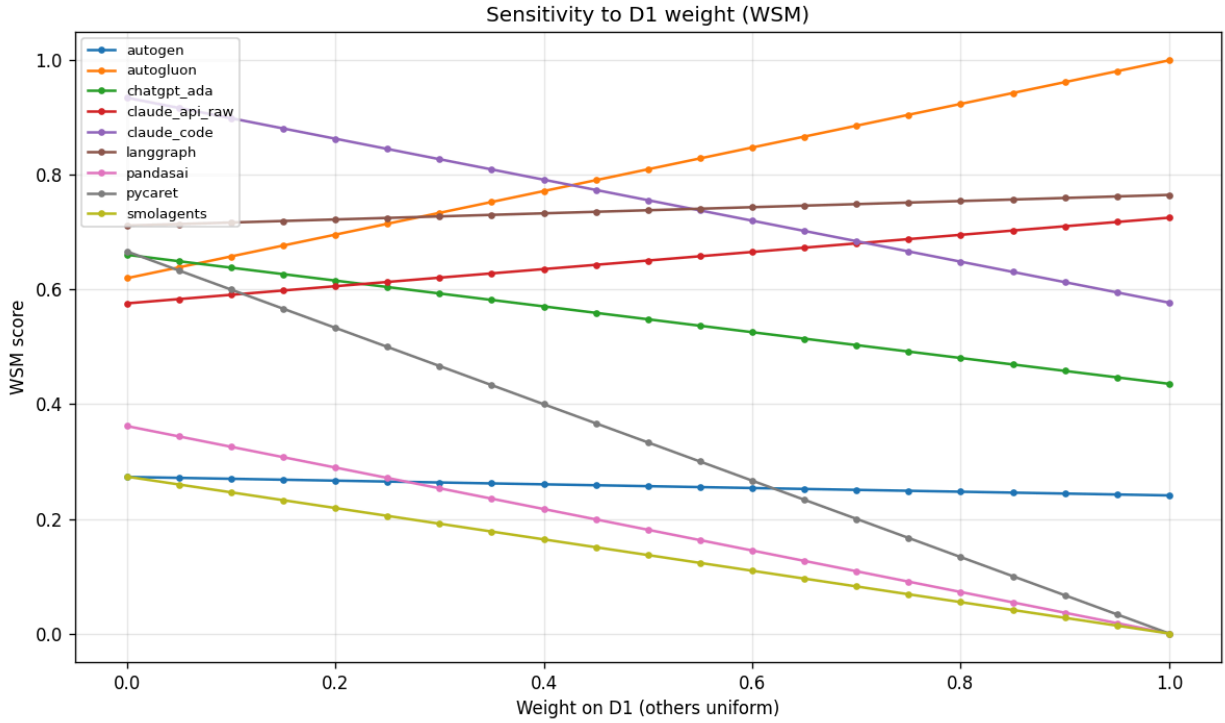


Figure 1. Sensitivity sweep on the D1 accuracy weight under WSM. The optimal agent flips from `claude_code` to `autogluon` at $w_{D1} = 0.45$, the lowest breakpoint among the seven dimensions.

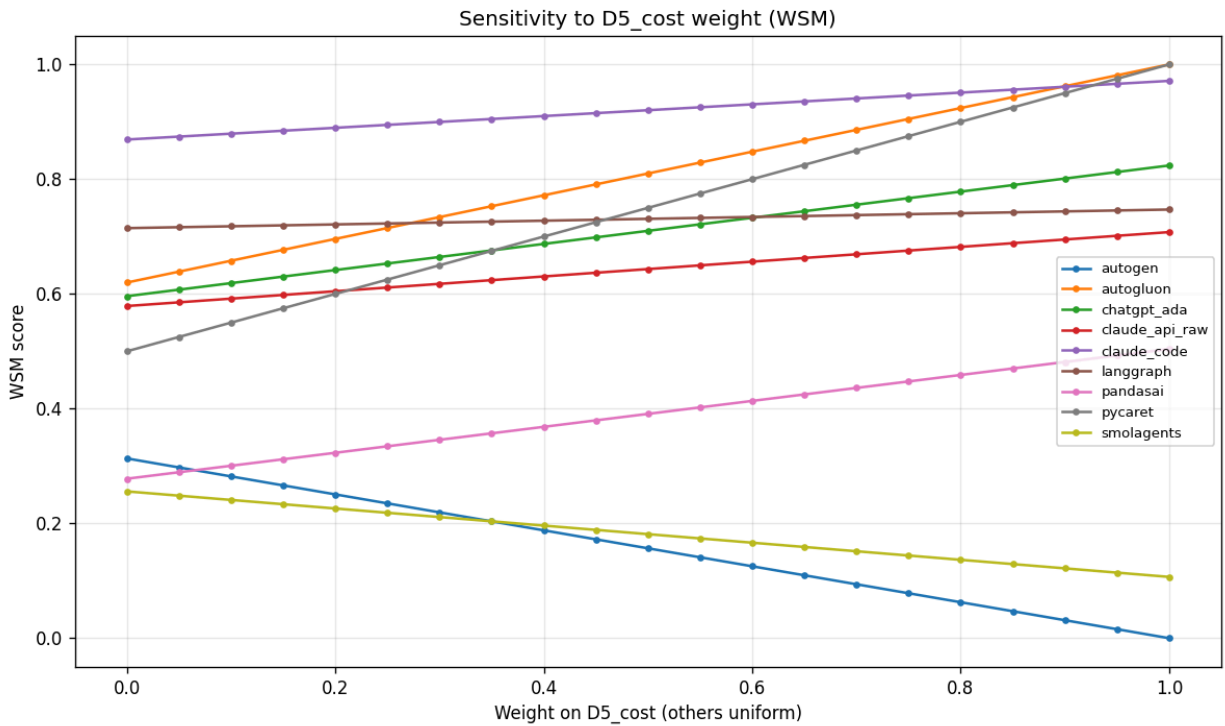


Figure 2. Sensitivity sweep on the D5 cost weight. `claude_code` remains optimal until $w_{cost} = 0.90$, after which `autogluon` dominates due to its zero per-run API cost.

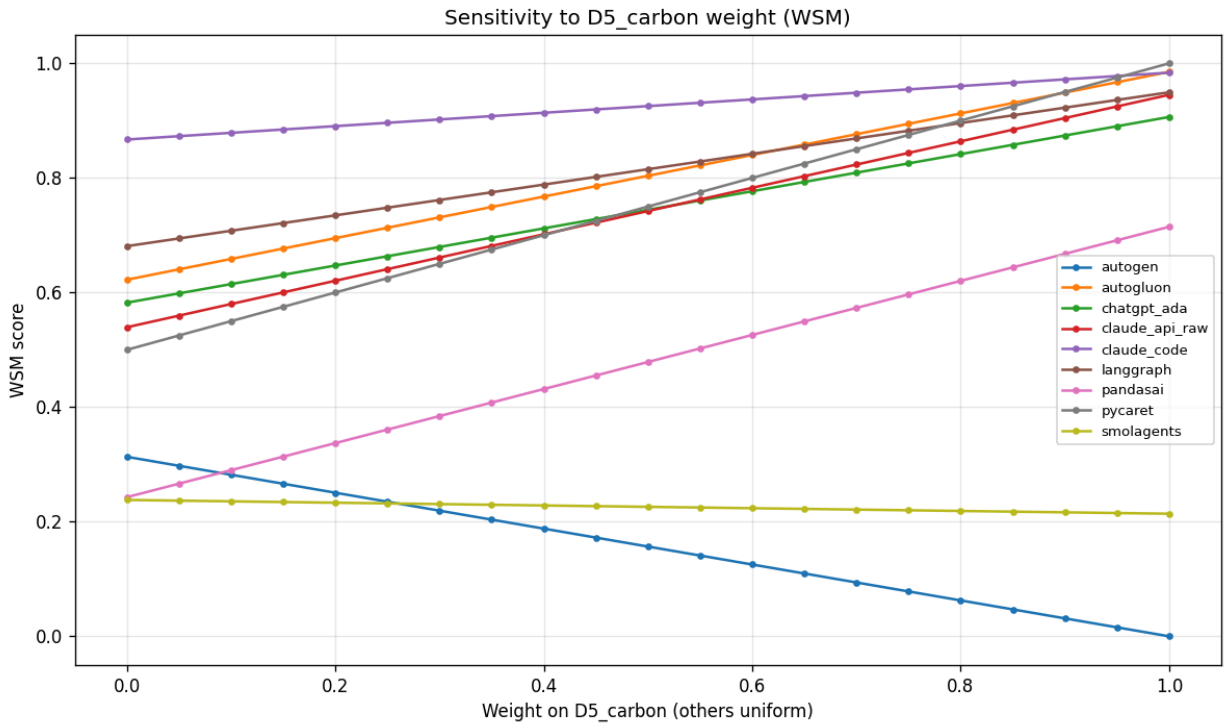


Figure 3. Sensitivity sweep on the D5 carbon weight. `claude_code` holds the optimum across nearly the entire range; `pycaret` only takes over at the extreme $w_{\text{carbon}} = 1.00$, confirming that agentic Claude is carbon-competitive with local AutoML.

4.4 Pareto frontier projections

Per-projection Pareto-optimal agents:

Projection	Pareto-optimal
Accuracy vs Cost	autogluon (alone)
Accuracy vs Carbon	autogluon, pycaret
Code Quality vs Cost	claude_code (alone)
Explainability vs Cost	claude_code (alone)
Cost vs Carbon	pycaret (alone)
Accuracy vs Speed	autogluon, langgraph, pycaret

AutoGluon dominates accuracy-vs-cost; Claude Code dominates the two code-quality projections; PyCaret owns the cost-vs-carbon corner. Three single-agent Pareto-optimum projections plus three multi-agent frontiers reveal the preference-driven nature of agent selection.

Figures 4 to 7 plot four representative 2D projections of the seven-dimensional scorecard. Each marker is one agent; orange filled markers are Pareto-optimal under that projection, grey

markers are dominated. The axes follow the convention "higher is better" for accuracy, code quality, and explainability; "lower is better" for cost, carbon, and speed.

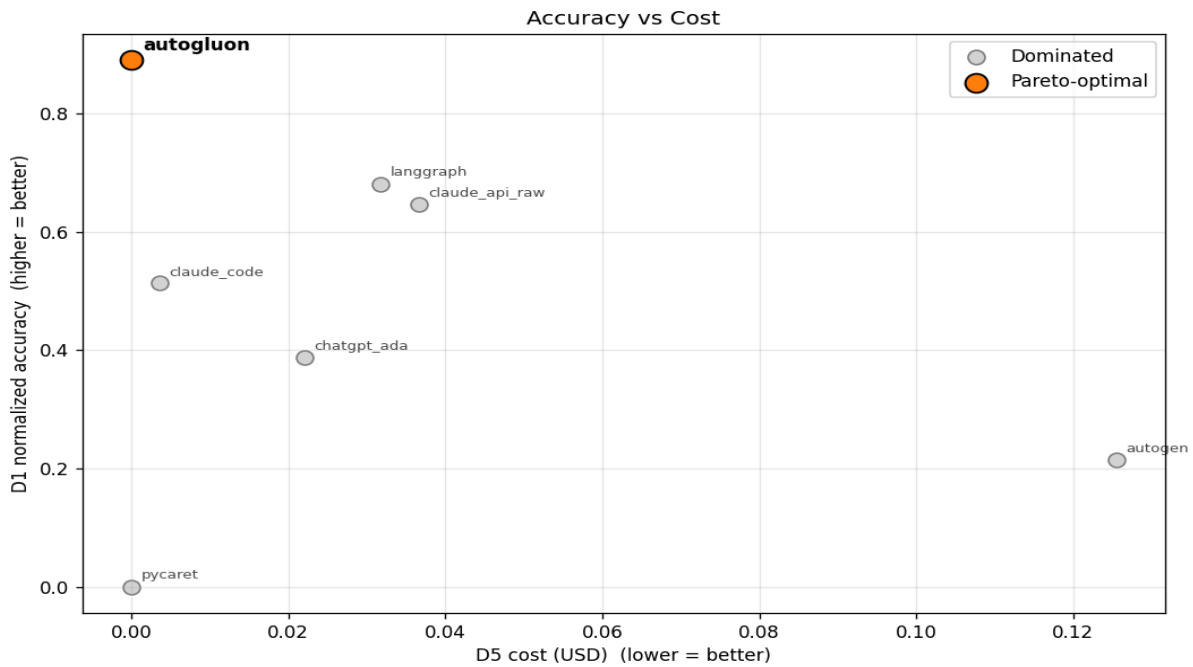


Figure 4. Pareto frontier in the accuracy-vs-cost projection. AutoGluon is the unique non-dominated agent: it achieves the highest normalized D1 score at zero per-run API cost.

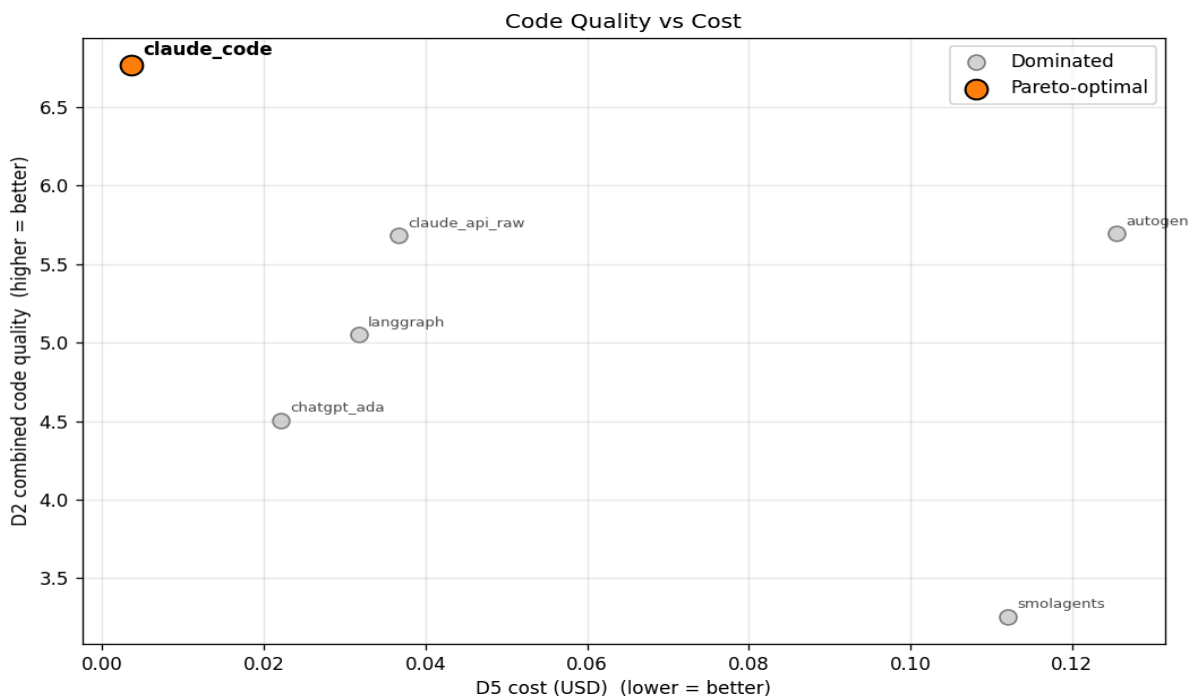


Figure 5. Pareto frontier in the code-quality-vs-cost projection. Claude Code is the unique non-dominated agent, owning the high-D2-low-cost corner.

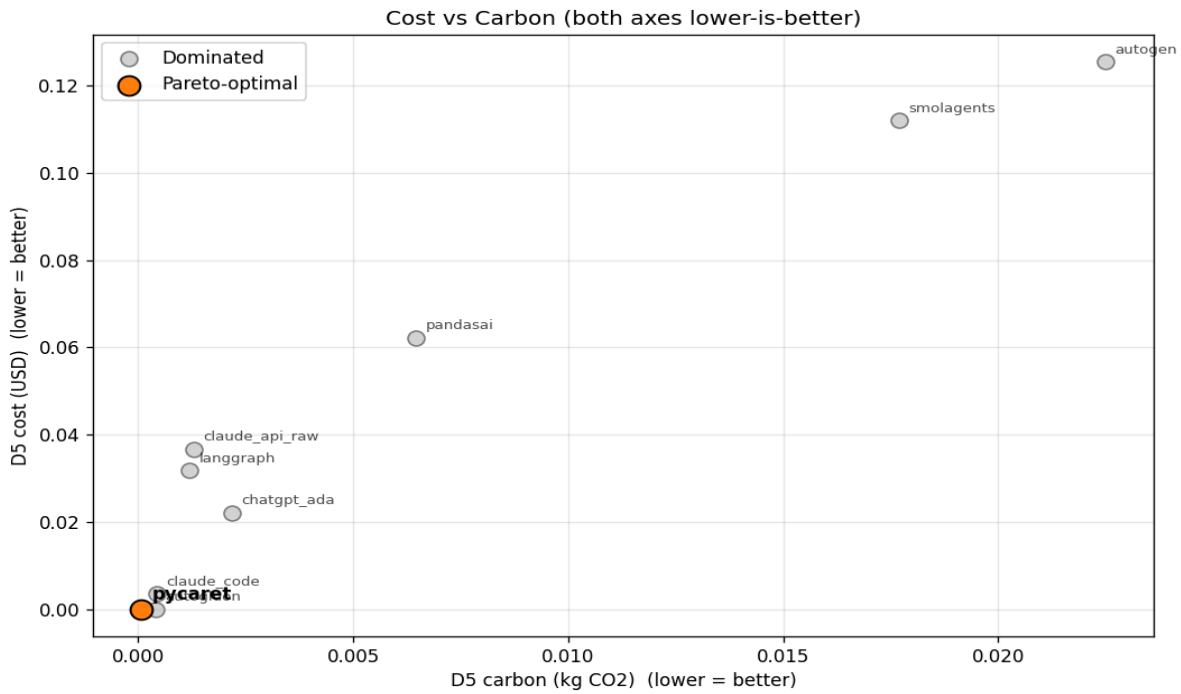


Figure 6. Pareto frontier in the cost-vs-carbon projection. PyCaret owns this corner: zero API cost combined with the lowest measured per-run carbon footprint among local AutoML agents.

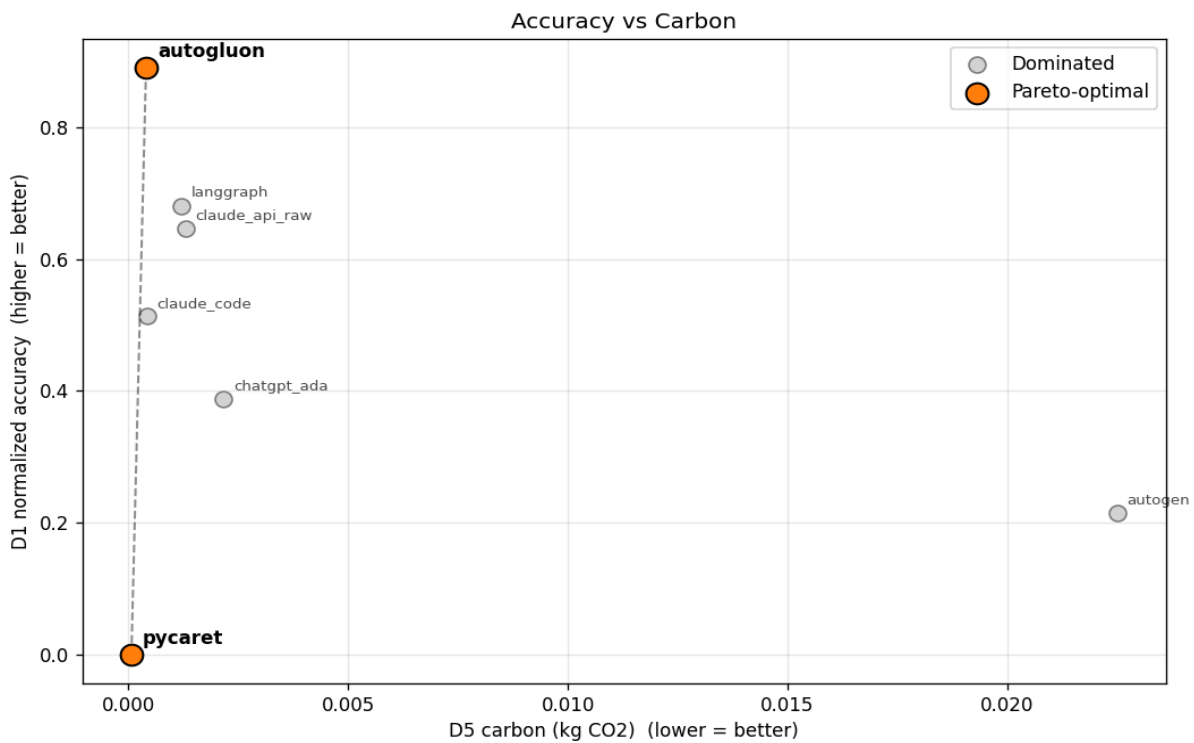


Figure 7. Pareto frontier in the accuracy-vs-carbon projection. Both AutoGluon and PyCaret are non-dominated; the routing choice between them depends on the D1-versus-D5 carbon trade-off ratio.

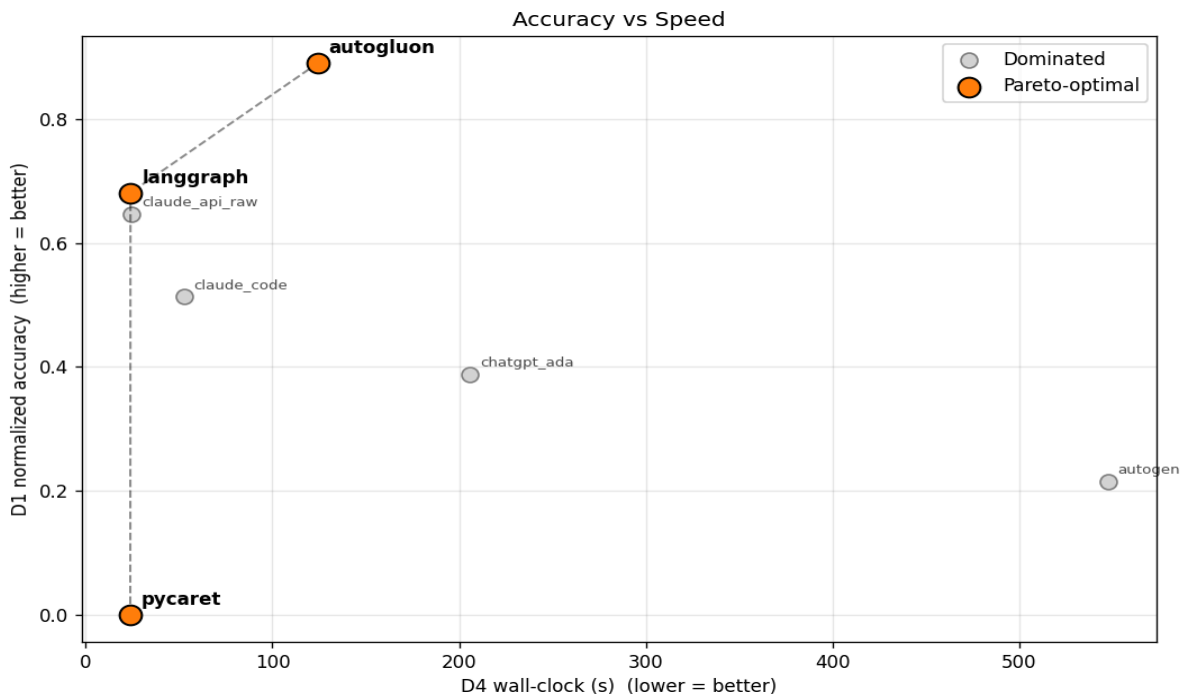


Figure 8 (Pareto: Accuracy vs Speed). Three-agent Pareto frontier on the accuracy-versus-speed plane: AutoGluon owns peak accuracy, LangGraph the speed-accuracy middle, PyCaret the speed corner. Five LLM agents are dominated.

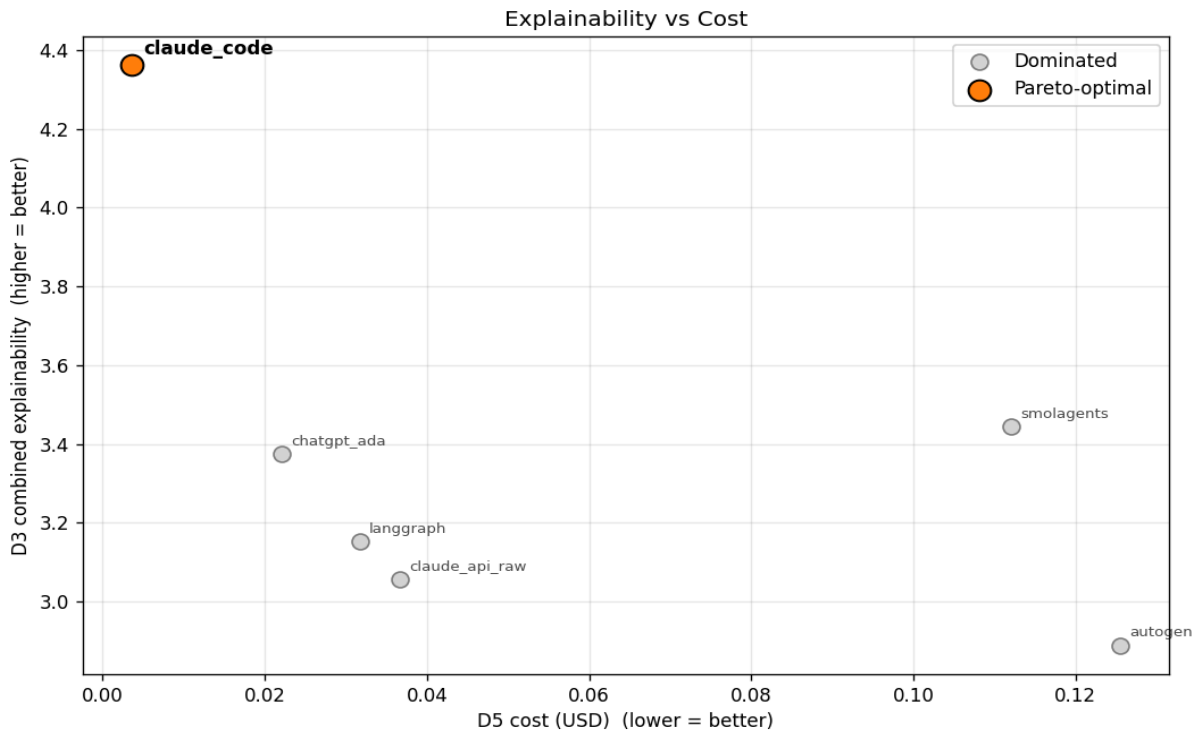


Figure 9 (Pareto: Explainability vs Cost). Pareto frontier in the explainability-versus-cost projection. Claude Code is the unique non-dominated agent, owning the high-D3-low-cost corner roughly 1.8× above every peer.

4.5 Statistical significance

Friedman + Nemenyi on D1 normalized primary metric, complete-coverage agent set (autogluon, claude_api_raw, claude_code, pycaret, n = 9 blocks):

- ❖ **Friedman:** $\chi^2 = 22.481$, $p = 5.18e-05$ (highly significant).
- ❖ **Nemenyi mean ranks:** autogluon 1.389, claude_api_raw 2.000, claude_code 2.611, pycaret 4.000.
- ❖ **Critical Difference at alpha = 0.05:** CD = 1.563.
- ❖ **Significant pairs:** pycaret vs autogluon, pycaret vs claude_api_raw.
- ❖ **Non-significant:** autogluon, claude_api_raw, claude_code are statistically indistinguishable on D1 across the three task types.

Wilcoxon signed-rank with Bonferroni correction on D5 cost (same agent set):

- ❖ All AutoML pairs differ significantly from raw API baselines.
- ❖ Claude Code vs Claude API raw on cost: 9 of 9 wins, $p_{\text{adj}} = 0.0234$, significant. The 10× cost ratio survives magnitude-based testing.

D5 carbon: claude_code is statistically indistinguishable from local agents (autogluon, pycaret) on emissions; claude_api_raw is significantly worse. This is a strong inferential result for the Green AI angle: agentic Claude achieves local-agent-level carbon emissions despite running on cloud LLM inference.

4.6 Multi-protocol study (Protocols A, B, C)

Twenty-one cells across three paradigm-locked agents under Protocol B (paradigm-native), versus their same-prompt-rule cells under Protocol A:

Agent	Protocol A	Protocol B
pandasai	0 of 27 cells (0 percent)	9 of 9 (100 percent)
smolagents	0 of 9 NLP cells (0 percent)	5 of 6 (83 percent)
langgraph	0 of 3 AR-ROBU cells (0 percent)	6 of 6 (100 percent)
total	0 of 39 (0 percent)	20 of 21 (95 percent)

Same data, same model family, same scoring. Only the prompt format changed. Success rate moves from 0 percent to 95 percent. The failure mode is paradigm-bound, not capability-bound. The single failure (smolagents AR-PRED run 3) hit the max_steps cap at 8, not a capability ceiling.

Sixty-three Protocol C cells (strict-explicit prompts on the seven agents that succeeded under Protocol A):

- ❖ Per-agent gap (C minus A) ranges from -0.145 (autogluon, worst) to $+0.196$ (autogen, best).
- ❖ Procrustes disparity = 0.412 (moderate displacement).
- ❖ Cross-protocol Kendall τ_{au} (A vs C) = 0.905 , $p = 0.003$ (highly correlated rankings, statistically significant).
- ❖ **Two-way ANOVA:** agent main effect $F = 12.15$, $p < 0.001$ (significant); protocol main effect $F = 0.25$, $p = 0.62$ (*NOT significant*); agent $\times$ protocol interaction $F = 0.52$, $p = 0.79$ (*NOT significant*).

Headline interpretation combining all three protocols: strict prompts redistribute, native paradigms transform. Within-class prompt variation (A vs C) produces small redistribution with no significant main effect; cross-class paradigm switching (A vs B) produces dramatic 0-to-95 percent transformation. Different prompt-protocol axes have qualitatively different effect magnitudes on agent performance.

4.7 Theoretical proposition

Proposition 1 (*WSM–PROMETHEE-II equivalence*)

Let $n \geq 2$ agents be evaluated on d dimensions, with min-max normalized scores $x_{a,i} \in [0, 1]$ and non-negative weights w_i satisfying $\sum_{i=1}^d w_i = 1$. Then for every weight vector and every score matrix, the rankings produced by the Weighted Sum Model (WSM) and by PROMETHEE-II with the linear preference function are identical.

Proof.

By definition, the WSM score of agent a is:

$$WSM(a) = \sum_{i=1}^d w_i x_{a,i}$$

The linear preference function on dimension i is:

$$P_i(x_{a,i}, x_{b,i}) = \max(0, x_{a,i} - x_{b,i}) ,$$

The aggregated pairwise preference is:

$$\Pi(a, b) = \sum_i w_i P_i(x_{a,i}, x_{b,i}) ,$$

and the PROMETHEE-II net flow is:

$$\varphi(a) = (1 / (n - 1)) \sum_{b \neq a} [\pi(a, b) - \pi(b, a)] .$$

For any real t , the identity $\max(0, t) - \max(0, -t) = t$ holds. Applied dimension-wise this gives

$$P_i(x_{a,i}, x_{b,i}) - P_i(x_{b,i}, x_{a,i}) = x_{a,i} - x_{b,i} ,$$

and therefore,

$$\pi(a, b) - \pi(b, a) = \sum_i w_i (x_{a,i} - x_{b,i}) .$$

Substituting into the net flow and writing $S_i = \sum_c x_{c,i}$ for the column sum on dimension i , we expand:

$$\sum_{b \neq a} (x_{a,i} - x_{b,i}) = (n - 1) x_{a,i} - (S_i - x_{a,i}) = nx_{a,i} - S_i ,$$

which yields

$$\varphi(a) = (n / (n - 1)) \cdot WSM(a) - C ,$$

where $C = (1 / (n - 1)) \sum_i w_i S_i$ is independent of the agent a .

Since $n / (n - 1) > 0$ for $n \geq 2$, the map $a \mapsto \varphi(a)$ is a positive affine transformation of $a \mapsto WSM(a)$. Two real-valued functions that differ by a positive affine transformation induce identical rankings.

Empirical confirmation. Kendall's τ between WSM and PROMETHEE-II rankings across all six routing presets is $\tau = 1.000$, consistent with the proposition.

Implication. TOPSIS, whose closeness coefficient depends non-linearly on the Euclidean distances to the ideal and anti-ideal points, does not collapse onto WSM under this setup. Empirically TOPSIS diverges from WSM with $\tau \in [0.833, 0.944]$ across the six presets. We therefore report WSM and PROMETHEE-II together as one equivalent strategy, and TOPSIS as the structurally independent second strategy.

4.8 Adversarial / D6 robustness

HD-ROBU-01 (binary classification under label flip plus missing-value injection) degradation per agent:

Agent	clean F1	adversarial F1	degrade
AutoGluon	0.9017	0.9017	0 percent
PyCaret	0.8793	0.8793	0 percent
LangGraph	0.9127	0.9182	−0.6 percent (within noise)
Claude API raw	0.9072	0.9006	0.7 percent
ChatGPT ADA	0.8962	0.8853	1.2 percent
Claude Code	0.9017	0.8908	1.2 percent
AutoGen	0.8899	0.8525	4.2 percent

AutoML agents are perfectly robust on this corruption. LLM agents are within 5 percent degradation. We acknowledge that label-flip-style adversarial protocols introduce a clean-vs-adversarial F1 comparison artifact (the adversarial test labels are also flipped, so a model that learns the flipped pattern scores high against the flipped test set); this is documented as a methodology limitation.

5. Discussion

5.1 RQ1 answer: heterogeneous agent profiles

Direct comparison across all six dimensions reveals heterogeneous agent profiles with no single dominator. AutoGluon excels at tabular accuracy and is perfectly robust on classification adversarial perturbations. Claude Code excels at code quality and explainability with very low cost and carbon. PyCaret excels at carbon efficiency and speed. LangGraph excels at HD-PRED classification narrowly. Each agent occupies a distinct corner of the seven-dimensional routing space.

5.2 RQ2 answer: no single dominator

Hypothesis confirmed at multiple levels. (1) In the Pareto analysis, seven of nine agents are non-dominated in at least one 2D projection. (2) In the routing analysis, the optimal agent flips between Claude Code, AutoGluon, and LangGraph across the six preset weight profiles. (3) In the sensitivity sweep, five of seven dimensions have a breakpoint where the optimal agent changes. PandasAI and smolagents are the only fully-Pareto-dominated agents, and Section 4.6 shows their dominance is paradigm-bound rather than capability-bound.

5.3 RQ3 answer: routing recommendations are preference-driven

PrismBench Router produces preset-driven optimal agents that flip with weight changes. Three MCDM methods (WSM, PROMETHEE-II, TOPSIS) are not three independent perspectives; Proposition 1 shows WSM and PROMETHEE-II are equivalent. TOPSIS provides the only second view that is not collinear with the first, with mild divergence (τ between 0.833 and 0.944) on five of six presets. Sensitivity sweeps quantify how robust each agent's routing position is to small weight changes.

5.4 RQ4 answer: scaffolding is efficiency, not accuracy

Claude Code vs Claude API raw is the cleanest controlled comparison for RQ4: same Sonnet model, scaffolding vs single-shot. Across three task types, F1 is within 3 percent (statistically indistinguishable under Wilcoxon and Nemenyi) but cost is 10× lower (Wilcoxon $p = 0.023$) and carbon is 3× lower. Code quality by static analysis is 60 percent higher; LLM-judge code quality is mixed (raw scores higher under judge on substantive structure, but combined score still favors Code).

The defensible RQ4 conclusion: agentic scaffolding's measurable benefit is token efficiency (and downstream cost and carbon savings), code structural maturity (by static analysis), and explainability quality (1.8× higher LLM-judge D3 score), not raw accuracy gain at the F1 level. ChatGPT ADA on the OpenAI side is a partial peer with scaffolding present but reliability inconsistencies on multi-step NLP tasks; we document this as an OpenAI Responses API limitation with extant continuation-loop mitigation.

5.5 Multi-protocol synthesis

Strict-explicit prompts (Protocol A vs C) produce small redistribution of agent performance with no significant main effect on the primary metric (ANOVA $p = 0.62$). Paradigm-native prompts (Protocol A vs B) produce dramatic transformation for paradigm-locked agents (0 percent to 95 percent success rate). The methodological implication for DS agent benchmarks is that prompt protocol is a routing-relevant user preference, not a benchmark artifact, and agents should be characterized along an interaction-style axis in addition to the six performance dimensions.

6. Limitations

1. **Sample size.** Three runs per (agent, task) cell is at the conservative floor for rank-based testing. Some pairwise effects (cost, carbon) recover significance only via Wilcoxon. With $n = 10$ per cell, Nemenyi CD shrinks substantially and several borderline pairs would gain inferential significance.
2. **Per-task NaN.** Three of nine agents (chatgpt_ada partial on AR; pandasai and smolagents on all task types under Protocol A) failed to produce a primary metric on at least one cell. We treat these as Pareto exclusions in routing rather than censoring agents from the study.

3. **Carbon precision.** Local agents are measured (CodeCarbon load-mode); cloud agents are estimated from token counts using a literature-based 0.5 kg per million tokens factor. Cross-category carbon comparisons are approximate; within-category comparisons are precise.
4. **LLM-as-Judge variance.** Per-cell Cohen kappa varies widely (-0.25 to $+0.67$) due to small sample size (5 dimension samples per cell). Judge scores are measurement artifacts, not consensus truth; we report kappa per cell for transparency.
5. **Label-flip artifact in AR-ROBU.** Adversarial test labels are flipped, so the clean-vs-adversarial F1 comparison can show spurious "improvement" for agents that fit the flipped training pattern. We document this as a methodology limitation.
6. **Single-machine evaluation.** Headline benchmark numbers were generated on Apple M4 host; the Docker artifact is verified to run the engine but not used for headline data generation. This is consistent with how AutoML and DS agent benchmarks typically validate Docker reproducibility.
7. **Agent versions are pinned snapshots.** Dependencies in requirements.in and agents.yaml reflect a 2026-05 snapshot. The agent ecosystem evolves rapidly; absolute numbers will shift with future releases.
8. **OpenAI Responses API output ceiling.** `claude_api_raw` hit a 4096-output-token ceiling on some longer NLP responses despite `max_tokens=8192` in the wrapper, suggesting an account-level or model-level cap beyond our control without an extended-output beta header.

7. Conclusion

PrismBench delivers a six-dimension benchmark of nine production-grade data science agents, a preference-aware MCDM routing framework with sensitivity and Pareto analyses, statistical validation via Friedman / Nemenyi / Wilcoxon, a multi-protocol study isolating prompt-protocol effects from agent capability, and a theoretical proposition characterizing the relationship between WSM and PROMETHEE-II.

The headline empirical finding is that no single agent dominates: seven of nine agents are Pareto-optimal in at least one preference profile. Claude Code is the robust default winner across the preference space at one-tenth the cost of equivalent-accuracy raw-LLM baselines. AutoGluon dominates tabular regression and accuracy-cost trade-offs. PyCaret owns cost-carbon efficiency. PandasAI and smolagents are paradigm-locked under benchmark conventions but succeed under matched paradigms, motivating an interaction-style preference dimension in the routing framework.

The headline methodological finding is that prompt protocol is a controlled experimental variable rather than an arbitrary fairness convention. Strict-explicit prompts redistribute performance without significantly changing main effects; paradigm-native prompts transform paradigm-locked

agents from 0 percent to 95 percent success. The artifact is open source and reproducible end-to-end inside a Docker image.

8. Future directions

The current PrismBench scorecard, routing framework, multi-protocol study, and theoretical proposition together constitute the core deliverable. Several extensions are configured but not yet executed, or are natural next steps that the methodology now makes tractable. We list them in priority order.

Figure B summarizes the work plan as a three-lane Gantt: the benchmark artifact lane is closed, the research-paper lane is mid-flight, and the productionization lane is scoped but not yet started. The red vertical line marks today (May 2026); items to its left are complete and items to its right are committed targets.

8.1 Causal scaffolding ablation for RQ4

Status: planned next. The Claude Code vs Claude API raw comparison establishes that agentic scaffolding delivers efficiency, code quality, and explainability gains at comparable accuracy. It does not isolate which scaffolding component is responsible.

A controlled ablation would hold the model fixed (claude-sonnet-4-6) and toggle four binary axes:

- ❖ **File-system access:** can the agent read and write workspace files?
- ❖ **Bash execution:** can the agent run shell commands and inspect output?
- ❖ **Multi-step iteration:** is the agent allowed to retry on failure?
- ❖ **Reflection:** does the agent self-critique before final submission?

The $2^4 = 16$ cells per task type, run three times each, would attribute observed efficiency gains to specific architectural primitives. This is the cleanest follow-up experiment the framework supports without engine changes, and is the headline experiment for the upcoming research paper.

8.2 Cross-modal extension

Status: scoped, partially configured. `configs/datasets.yaml` already declares image (CIFAR-10) and audio (UrbanSound8K) tasks; only the agent-side modality wiring is missing. The benchmark's same-prompt rule and seven-dimensional scorecard generalize without modification.

The open empirical question: whether PandasAI and AutoGluon-Tabular are excluded by data-type ineligibility, or whether modality routing produces cleaner Pareto frontiers because

agents stratify more sharply by input type. A parallel cross-modal suite would extend the routing framework from tabular and NLP into the full data-science task space.

8.3 Routing under preference uncertainty

Status: in progress (paper section). The current router takes a single weight vector w as input. In practice, a user's preferences are themselves uncertain. Two complementary formulations suggest themselves:

Bayesian routing: place a Dirichlet prior over w , marginalize the MCDM score, and report the posterior probability that each agent is optimal. This converts a point recommendation into a calibrated belief and exposes which agents are robust default picks across plausible weight regions.

Robust routing: solve a min-max problem of the form $\min_{\text{agents}} \max_{w \in U} \text{regret}(\text{agent}, w)$, where U is an uncertainty set around a nominal weight. This produces an agent that minimizes worst-case regret over the user's stated uncertainty, a stronger guarantee than expected-utility routing.

Both formulations build on the existing scorecard and require only post-processing of the same MCDM machinery; preliminary derivations are drafted in `notes/findings_routing.md`.

8.4 Larger sample sizes and Critical Difference shrinkage

Status: planned next, compute-bound. Three runs per cell is the conservative floor for rank-based testing. Scaling to ten runs per cell shrinks the Nemenyi Critical Difference at $\alpha = 0.05$ from 1.563 to roughly 0.860 and would recover inferential significance for several adjacent rank pairs that are currently borderline (`claude_code` vs `claude_api_raw` on D1; `autogluon` vs `langgraph` on HD-PRED). This is a pure compute scale-up with no methodological change, gated on cloud-credit availability for the LLM agents.

8.5 Train-clean / test-perturbed adversarial protocol

Status: planned next. The label-flip artifact in HD-ROBU and AR-ROBU is a known limitation of adversarial benchmark design (Section 6, item 5). The natural correction is the train-clean / test-perturbed protocol: agents are shown the unperturbed training data, and only the test inputs are corrupted with bounded missing-value injection or feature noise.

This removes the spurious "improvement" pathway and yields a clean distributional-shift robustness measurement. Implementation requires an `adversarial_test_only=True` flag in `src/data_loader.py` and parallel ROBU-V2 task definitions; both are small additions.

8.6 Live continuously-updated leaderboard

Status: scoped, productionization path. The current artifact is a snapshot at the 2026-05 dependency versions in `requirements.in` and `agents.yaml`. The agent ecosystem moves on a quarterly cycle.

A live leaderboard, re-running the 81-cell main benchmark on each agent version bump and publishing the updated PrismBench scorecard, would convert the static artifact into a longitudinal resource. The infrastructure for this is largely the existing `python -m src.run_benchmark` plus `python -m src.scorecard` pipeline, wrapped in CI on a monthly cron with results posted as a GitHub Pages site.

8.7 Interaction-style preference dimension

Status: in progress (paper section). The multi-protocol study (Section 4.6) shows that paradigm fit transforms paradigm-locked agents from 0 percent to 95 percent success rate. This argues for adding an interaction-style axis to the routing framework:

- ❖ A user who prefers conversational queries should be routed to PandasAI.
- ❖ A user who prefers ReAct-style step decomposition should be routed to LangGraph.
- ❖ A user who prefers code-iterative workflows should be routed to smolagents.

This holds regardless of the seven performance dimensions. The MCDM machinery accommodates a categorical “compatibility” weight without modification; the open work is defining the compatibility scoring rubric per agent and validating it against user-study data.

9. References

- Brans, J.-P. & Vincke, P. (1985). "A Preference Ranking Organisation Method: The PROMETHEE Method for MCDM." *Management Science*, 31(6), 647–656
- Demsar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.
- Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., Smola, A. (2020). AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. *arXiv:2003.06505*.
- Hwang, C.-L., Yoon, K. (1981). *Multiple Attribute Decision Making: Methods and Applications*. Springer.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*.
- Jing, L., Huang, Z., Wang, X., Yao, W., Yu, W., Ma, K., Zhang, H., Du, X., Yu, D. (2025). DSBench: How Far Are Data Science Agents from Becoming Data Science Experts? *ICLR*.
- Lacoste, A., Luccioni, A., Schmidt, V., Dandres, T. (2019). Quantifying the Carbon Emissions of Machine Learning. *Workshop on Tackling Climate Change with Machine Learning, NeurIPS*.
- Lai, E., Vitagliano, G., Zhang, Z., Chabra, O., Sudhir, S., Zeng, A., et al. (2025). KramaBench: A Benchmark for AI Systems on Data-to-Insight Pipelines over Data Lakes. *arXiv:2506.06541*.

- Luccioni, A. S., Viguier, S., Ligozat, A.-L. (2023). Estimating the Carbon Footprint of BLOOM, a 176B Parameter Language Model. *Journal of Machine Learning Research*.
- OpenAI (2024), Chan, J. S., Chowdhury, N., Jaffe, O., et al. (2024) . MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. *arXiv:2410.07095*.
- Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L.-M., Rothchild, D., So, D., Texier, M., Dean, J. (2021). Carbon Emissions and Large Neural Network Training. *arXiv:2104.10350*.
- Strubell, E., Ganesh, A., McCallum, A. (2019). Energy and Policy Considerations for Deep Learning in NLP. *ACL*.

10. Appendix

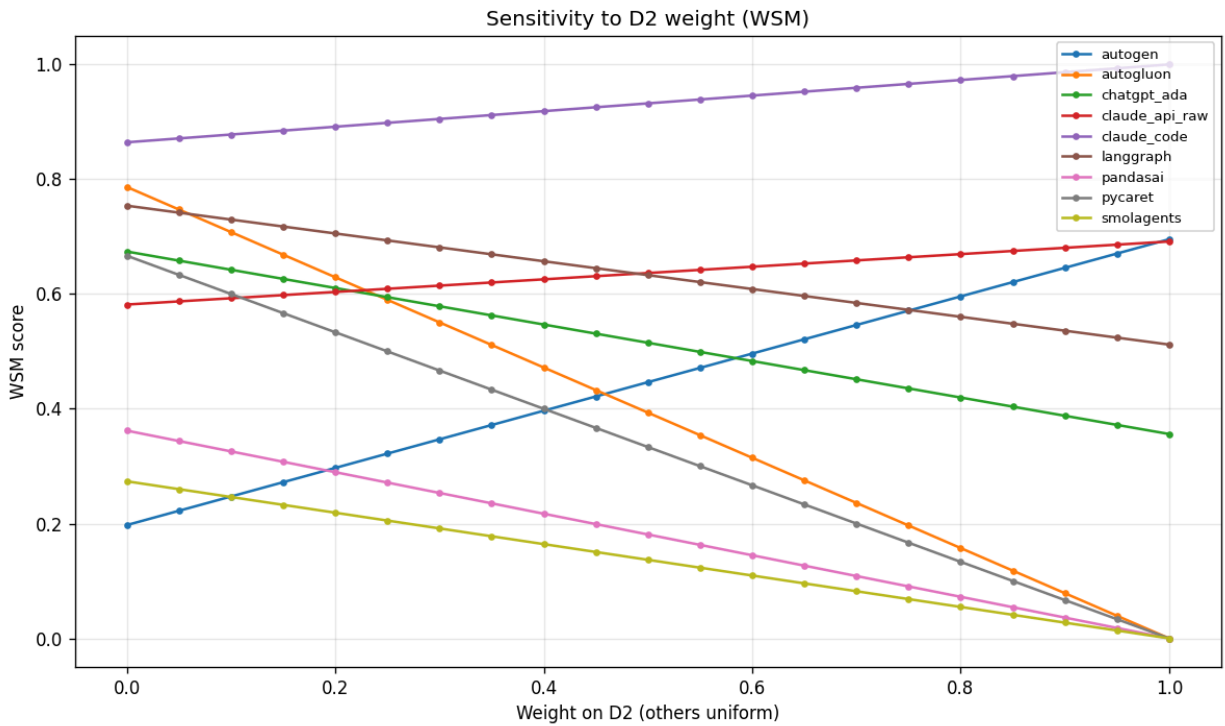


Figure (Sensitivity: D2 code quality). Sensitivity sweep on the D2 code-quality weight. Claude Code holds the optimum across the full weight range with no breakpoint, lead widening monotonically toward $w = 1$.

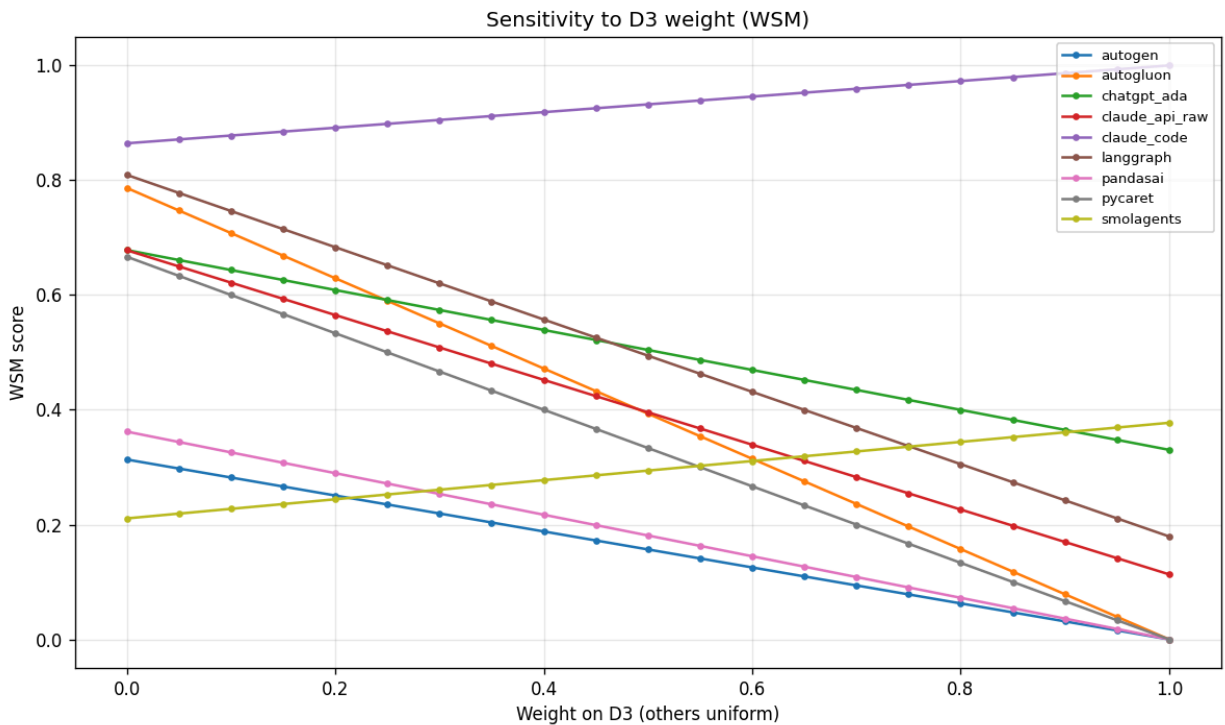


Figure (Sensitivity: D3 explainability). Sensitivity sweep on the D3 explainability weight. Claude Code dominates the full weight range with no breakpoint; gap to the next-best agent grows monotonically with weight.

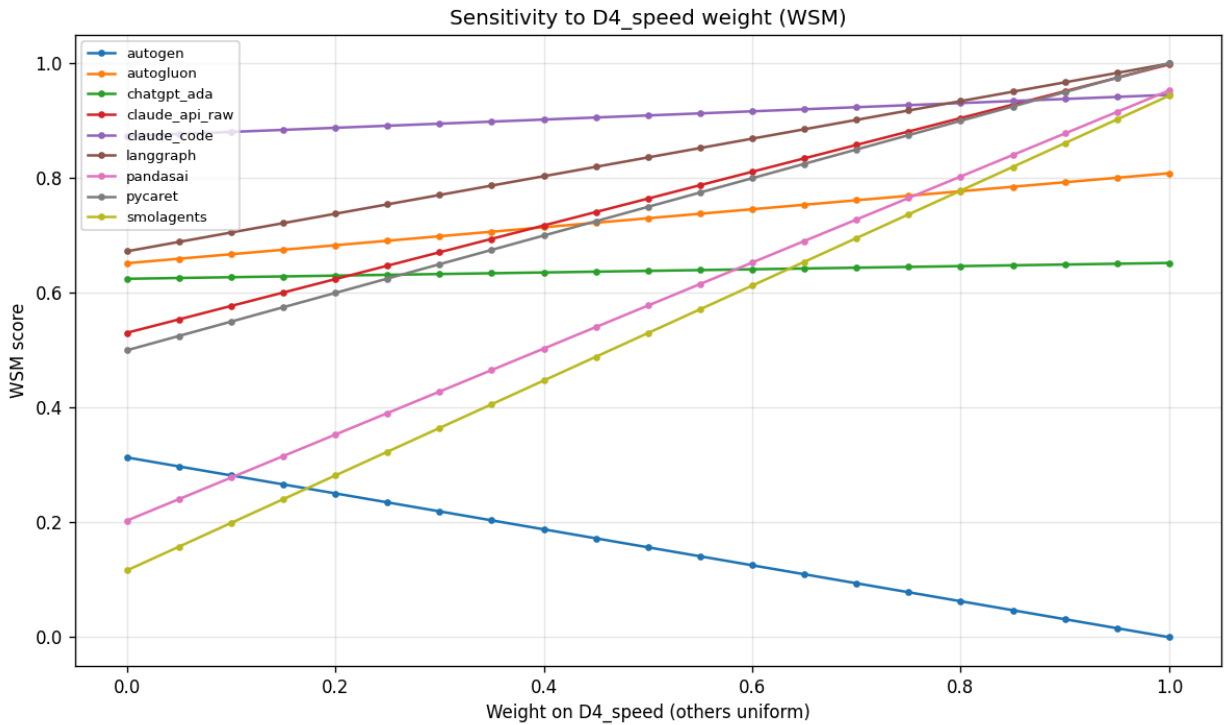


Figure (Sensitivity: D4 speed). Sensitivity sweep on the D4 speed weight. Claude Code wins until $w \approx 0.80$ where LangGraph takes over; at $w = 1.00$ PyCaret's ~6s wall-clock wins.

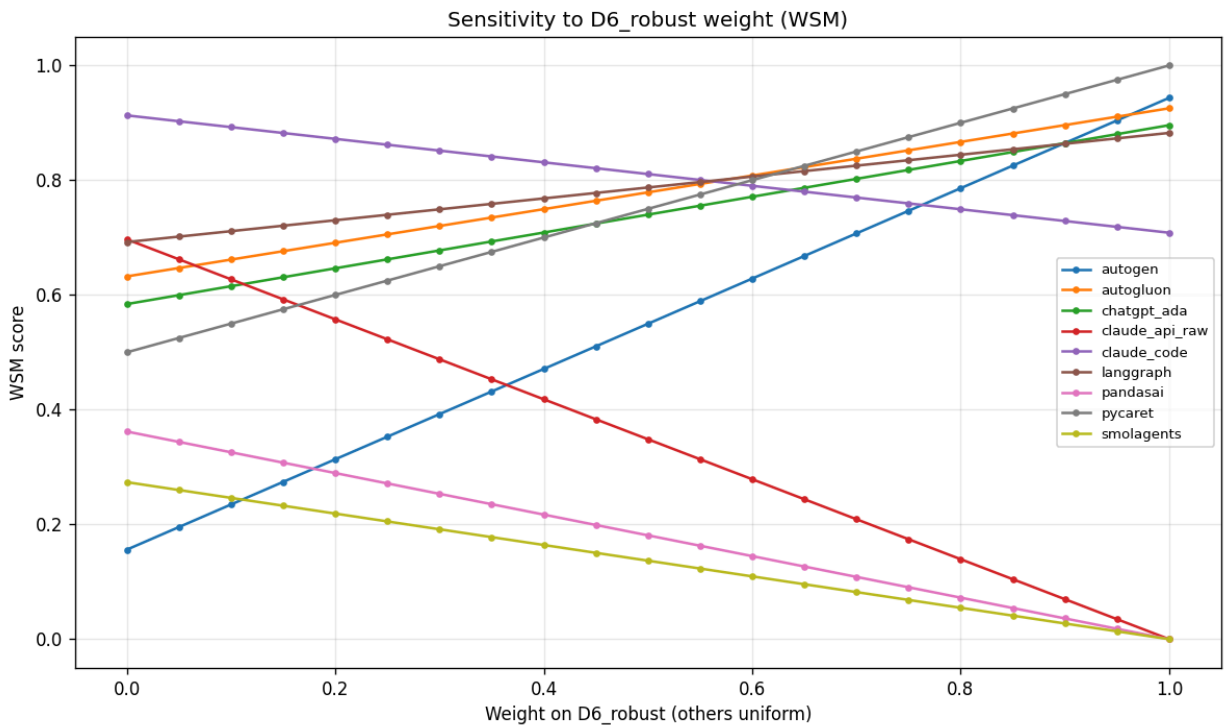


Figure (Sensitivity: D6 robustness). Sensitivity sweep on the D6 robustness weight. Two breakpoints appear: `claude_code` flips to `autogluon` at $w \approx 0.60$, then to `pycaret` at $w \approx 0.65$.